



Os Command Injection

- 
- > Os Command Injection
 - > Blind Os Command Injection
 - > Herramientas

- > Os Command Injection
- > Blind Os Command Injection
- > Herramientas

Introducción

- > La inyección de comandos del sistema operativo (también conocida como **inyección de shell**) es una vulnerabilidad que permite a un atacante ejecutar comandos arbitrarios del sistema operativo (SO) en el servidor que ejecuta una aplicación.
- > Por lo general, explotando esta vulnerabilidad, se compromete completamente la aplicación y todos sus datos.
- > Además, un atacante podría aprovechar esta vulnerabilidad para comprometer otras partes de la infraestructura que tengan relaciones de confianza con la app.
- > A esto se lo llama movimiento lateral

Ejecutando un ataque

> Consideremos una aplicación de compras que le permite al usuario ver si un artículo está disponible en una tienda en particular.

> Se accede a esta información a través de la siguiente URL:

```
https://insecure-website.com/stockStatus?productID=381&storeID=29
```

> Para proporcionar la información de stock, la aplicación debe consultar varios sistemas heredados.

> Por razones históricas, la funcionalidad se implementa llamando a un comando de shell con los ID de producto y tienda como argumentos:

```
stockreport.pl 381 29
```

Ejecutando un ataque

> Este comando genera el estado de existencias para el artículo especificado, que se devuelve al usuario.

> Si la app no controla la inyección de comandos, un atacante podría enviar la siguiente entrada:

```
& echo aiwefwlguh &
```

> Si esta entrada se envía en el parámetro productId, entonces el comando ejecutado por la aplicación es:

```
stockreport.pl & echo aiwefwlguh & 29
```

Ejecutando un ataque

- > El comando **echo** imprime un string de entrada en la salida y es una forma útil de probar algunos tipos de inyección de comandos.
- > El carácter **&** es un separador de comandos de shell.
- > Por consiguiente, el payload envía tres comandos separados uno tras otro.
- > Como resultado, la salida devuelta al usuario es:

```
Error - productID was not provided  
aiwefwlguh  
29: command not found
```

Ejecutando un ataque

- > Las tres líneas de salida demuestran que:
 - El script **stockreport.pl** original se ejecutó sin los argumentos esperados y, por lo tanto, devolvió un mensaje de error.
 - Se ejecutó el comando **echo** inyectado y la cadena proporcionada se repitió en la salida.
 - El argumento **29** se ejecutó como un comando, lo que provocó un error.
- > Colocar el separador de comando **&** después del comando inyectado es útil porque separa el comando inyectado de lo que sigue al punto de inyección.

Comandos útiles

- > Cuando identificamos esta vulnerabilidad generalmente es útil ejecutar algunos comandos iniciales para obtener información sobre el sistema que ha comprometido.
- > A continuación, se muestra un resumen de algunos comandos que son útiles en las plataformas Linux y Windows:

Propósito	Linux	Windows
Nombre de usuario	whoami	whoami
Sistema Operativo	uname -a	ver
Configuración de red	ifconfig	ipconfig /all
Conexiones de red	netstat -an	netstat -an
Procesos corriendo	ps -ef	tasklist

- > Os Command Injection
- > Blind Os Command Injection
- > Herramientas

Introducción

- > Ocurren cuando la aplicación **no devuelve** la salida del comando dentro de su respuesta HTTP.
- > Las vulnerabilidades **ciegas** siguen siendo explotables pero se requieren de técnicas diferentes.
- > Pensemos en un sitio web que permita a los usuarios enviar comentarios sobre el sitio.
- > El usuario ingresa su dirección de correo electrónico y su mensaje.
- > La aplicación del lado del servidor luego genera un correo electrónico a un administrador del sitio que contiene los comentarios.

Ejemplo

> Para hacer esto, llama al programa mail con los parámetros correspondientes.

```
mail -s "Este sitio esta buenisimo" -a  
From:he-man@grayskull.com  
feedback@vulnsite.com
```

> La salida del comando **mail** (si corresponde) no se devuelve en las respuestas de la aplicación, por lo que el uso del payload echo anterior no sirve..

> Deberemos utilizar otras técnicas para detectar y explotar esta vulnerabilidad

Detectando Command OS basado en tiempo

- > Podemos inyectar un comando que active un retraso de tiempo, lo que le permitirá confirmar que el comando se ejecutó en función del tiempo que tarda la aplicación en responder.
- > El comando ping es una forma eficaz de hacer esto, ya que le permite especificar la cantidad de paquetes ICMP a enviar y, por lo tanto, el tiempo que tarda el comando en ejecutarse:

```
& ping -c 10 127.0.0.1 &
```

- > Este comando hará que la aplicación envíe 10 paquetes (uno por segundo) ICMP a su adaptador de red.
- > Esto mantendrá en espera a la aplicación durante 10 segundos

Explotando OS Command Injection

- > Puede redirigir la salida del comando inyectado a un archivo dentro de la carpeta raíz de la aplicación web.
- > Luego puede recuperar esa información usando su navegador.
- > Por ejemplo: si la aplicación almacena recursos estáticos en la ubicación **/var/www/static** podemos enviar el siguiente payload:

```
& whoami > /var/www/static/whoami.txt &
```

- > El caracter > envía la salida del comando whoami al archivo especificado.
- > A continuación, puede utilizar su navegador para recuperar el archivo y ver la salida del comando inyectado en <https://vulsite.com/whoami.txt>

Formas de inyectar comandos

- > Se pueden usar diferentes metacaracteres de la consola para realizar ataques.
- > Algunos caracteres funcionan como separadores de comandos, lo que permite encadenar los comandos.
- > Los siguientes separadores de comandos funcionan tanto en sistemas basados en Windows como en Unix:

&	;
&&	0x0a o \n

Formas de inyectar comandos

- > En los sistemas basados en Unix, se puede usar **comillas invertidas** o el carácter **\$** para ejecutar un comando inyectado dentro del comando original:
 - ``comando inyectado``
 - `$(comando inyectado)`
- > Tener en cuenta que los diferentes metacaracteres pueden comportarse de forma diferente en determinadas situaciones.
- > A veces, la entrada que estamos explotando puede estar entre comillas en el comando original.
- > En este caso, deberemos terminar el contexto entre comillas (usando `"` o `'`) antes de usar los metacaracteres de consola adecuados para inyectar un payload.

Formas de prevenir la inyección de comandos

- > La forma más eficaz de prevenir estas vulnerabilidades es nunca llamar a los comandos del sistema operativo desde el código de la capa de aplicación.
- > En prácticamente todos los casos, existen formas alternativas de implementar la funcionalidad requerida utilizando APIs más seguras.
- > Si es inevitable entonces se debe realizar una validación de entrada robusta:
 - Validación con una lista blanca de valores permitidos.
 - Validando el tipo de datos y el contenido.
- > No intente controlar la entrada escapando los metacaracteres.
- > Es propenso a errores y vulnerable a ser bypassado por un atacante experto.

- > Os Command Injection
- > Blind Os Command Injection
- > Herramientas



Netcat

- > netcat es probablemente la herramienta más simple y más útil que se encuentra disponible para interactuar con sistemas en red.
- > Permite a un usuario mover datos por la red emulando el comando cat de linux/unix.
- > Permite enviar la información a través de una red TCP o UDP utilizando números de puertos determinados por el usuario.
- > Originalmente desarrollada en 1996. En 1998 Weld Pond creó la versión para windows.
- > Existe también una versión que encripta los datos enviados denominada cryptcat.

Cómo funciona Netcat

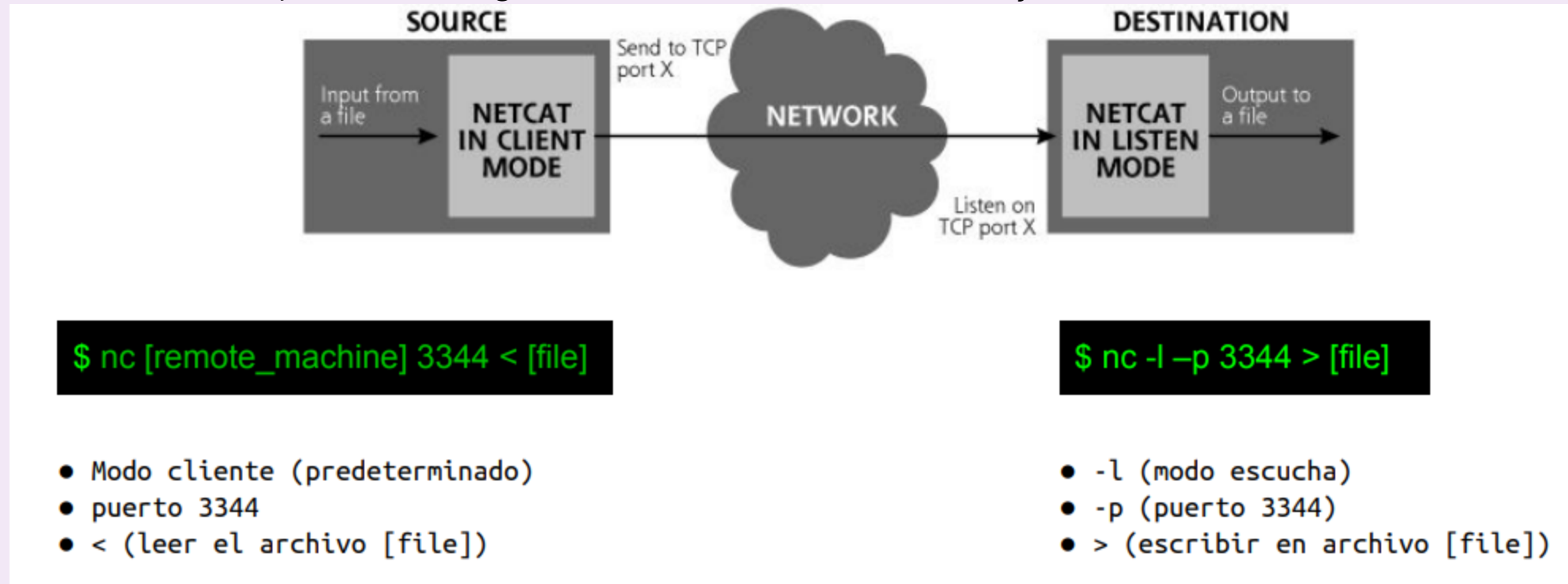
- > La aplicación netcat opera de dos modos posibles: **modo escucha** y **modo cliente**.
 - **Modo cliente**: se utiliza para iniciar una conexión TCP o UDP contra otra computadora. Netcat toma la información de la entrada estándar (teclado, archivo, etc.) y la envía por la red al destino.
 - **Modo escucha**: abre un puerto TCP o UDP y espera que llegue información por dicho puerto. En este modo, netcat envía los datos recibidos a la salida estándar (pantalla, archivo, etc.)

Transferencia de archivos con Netcat

- > Una de las formas más simples de utilizar netcat es para transferir archivos entre dos computadoras.
- > En la mayoría de las redes, el tráfico FTP entrante y saliente está bloqueado, motivo por el cual un atacante no podría utilizar estos servicios.
- > Pero si un atacante logra instalar netcat en un sistema podría utilizarlo para transferir archivos desde y hacia ese sistema utilizando cualquier puerto TCP o UDP que se encuentre permitido por el firewall.
- > Hay dos formas de utilizar netcat para transferir archivos:
 - Haciendo un **push** desde el cliente al listener
 - Haciendo un **pull** desde el listener hacia el cliente.

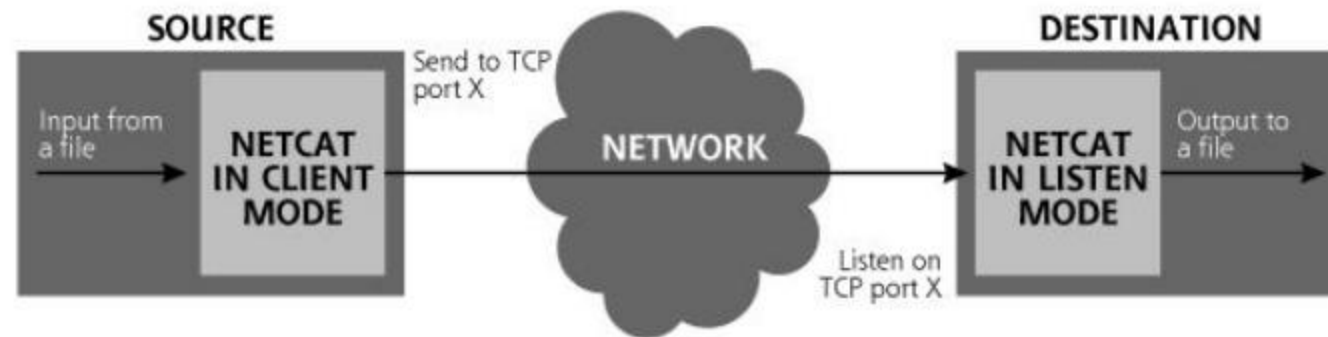
Push de archivos con Netcat

> Con el método push el origen se conecta al destino y envía el archivo



Pull de archivos con Netcat

> Con el método pull el destino se conecta hacia el origen, y una vez que están conectados, comienza a bajarse el archivo desde el origen hacia el destino.



```
nc -l -p 3344 < [file]
```

- -l (modo escucha)
- -p (puerto 3344)
- < (enviar archivo por la red)

```
$ nc [remote_machine] 3344 > [file]
```

- Modo cliente (predeterminado)
- -p (puerto 3344)
- > (escribir en el archivo [file])

Escanear puertos con Netcat

- > Si bien existen herramientas específicas mucho mejores (tales como **nmap**), con netcat también se pueden efectuar escaneos de puertos.
- > Solo permite hacer escaneos de puertos con el método **connect**: es decir, con los tres pasos de inicio de conexión TCP completos.

```
$ echo QUIT | nc -v -w3 [target] [startport]-[endport]
```

- > El comando anterior ejecuta un escaneo de puertos entre los puertos [startport] y [endport] e imprime el comando quit en caso que se conecte a algún servicio.
- > La opción -v se utiliza para dar verbosidad a la salida por pantalla.
- > La opción -w3 limita el tiempo de espera a 3 segundos.

Dudas?

